

CONTINUACIÓN: CAPÍTULO IV.

DESCRIPCIÓN DE LA PROPUESTA

4.2 Requerimientos de Hardware

Para garantizar el correcto funcionamiento, despliegue y consumo de la plataforma web de solicitudes de compra y venta, se definieron los requerimientos mínimos y recomendados de hardware. Estos se dividen en el entorno del servidor (donde se aloja el Backend en Node.js y el motor de base de datos) y el entorno del cliente (las estaciones de trabajo de las 32 gerencias, compras, ventas y almacén).

Entorno	Componente de Hardware	Requisito Mínimo	Requisito Recomendado
Servidor de Aplicaciones y Base de Datos	Procesador (CPU)	Intel Xeon o AMD EPYC de 4 núcleos (o equivalente)	Intel Xeon o AMD EPYC de 8 núcleos o superior
	Memoria RAM	8 GB DDR4 / DDR5	16 GB DDR4 / DDR5 o superior
	Almacenamiento (Disco)	120 GB en SSD (Estado Sólido)	240 GB o superior en SSD NVMe (Configuración RAID)
Estaciones de Cliente (32 Gerencias / Logística)	Procesador (CPU)	Intel Core i3 / AMD Ryzen 3 o superior	Intel Core i5 / AMD Ryzen 5 o superior
	Memoria RAM	4 GB DDR3 / DDR4	8 GB DDR4 o superior
	Conectividad	Tarjeta de Red LAN 100 Mbps Ethernet	Tarjeta de Red GigaLAN (10/100/1000 Mbps)

4.3 Requerimientos de Software

El entorno lógico del sistema web aprovecha el software libre para el desarrollo y despliegue, reduciendo costes de licenciamiento a CVG CABELUM y asegurando compatibilidad multiplataforma a través de estándares web internacionales.

Entorno	Componente Lógico	Especificación Tecnológica Seleccionada
Entorno del Servidor	Sistema Operativo	Linux Ubuntu Server LTS o Enterprise RedHat / Rocky Linux
	Entorno de Ejecución Backend	Node.js (Versión Long Term Support - LTS) con Express.js
	Motor de Base de Datos	RDBMS Relacional SQL (MySQL Server / PostgreSQL)
	Servidor Web Inverso	NGINX o Apache HTTP Server (Para proxy y certificados SSL/TLS)
Entorno del Cliente	Sistema Operativo Cliente	Microsoft Windows 10/11, GNU/Linux o macOS
	Navegadores Web Compatibles	Google Chrome, Microsoft Edge, Mozilla Firefox, Opera (Versiones modernas)

4.4 Requerimientos Funcionales

De acuerdo con la norma IEEE 830, los requerimientos funcionales definen los servicios esenciales que el sistema debe ejecutar de forma obligatoria ante las entradas y acciones de los usuarios. Para el flujo logístico de CVG CABELUM se establecieron los siguientes requerimientos específicos:

- **RF-01 Autenticación Basada en Roles:** El sistema debe restringir el acceso a los módulos mediante credenciales de usuario y clave cifrada, asignando dinámicamente un token de sesión (JWT) y el perfil correspondiente (Coordinador, Gerente, Compras, Almacén, TI).
- **RF-02 Creación y Registro de Solicitudes:** La plataforma debe permitir a los Coordinadores de Área abrir formularios digitales para introducir solicitudes de compra o de venta, detallando ítems, cantidades, justificación de área, prioridad y adjuntos técnicos.
- **RF-03 Control de Flujo Jerárquico:** El sistema debe transicionar el estatus de la solicitud de forma automática. Al ser guardada, quedará en estado "Pendiente por Aprobación de Gerencia", notificando visualmente al Gerente respectivo en su panel de control.
- **RF-04 Aprobación y Validación Operativa:** El módulo de Gerente debe permitir la opción de aprobar el requerimiento (enviándolo a Almacén/Compras) o rechazarlo añadiendo un campo obligatorio de observaciones o motivos del rechazo.
- **RF-05 Cruce de Inventario (Almacén):** El sistema debe proveer una interfaz dedicada para el Analista de Almacén, permitiendo marcar si el insumo solicitado cuenta con stock en planta o si requiere necesariamente ser procesado para compra externa.
- **RF-06 Procesamiento e Historial de Procura:** El perfil de Gestor de Compras debe poder registrar las actualizaciones logísticas, tales como asignación de proveedor y número de orden de compra, cerrando el ciclo al cambiar el estatus a "Procesado".
- **RF-07 Generación de Reportes Dinámicos:** El sistema debe compilar los datos transaccionales para emitir reportes cuantitativos detallados sobre el volumen de solicitudes por gerencia, tiempos de respuesta y costos estimados, permitiendo descargas en formatos estándares (PDF/CSV).

4.5 Requerimientos No Funcionales

Los requerimientos no funcionales definen los atributos de calidad, propiedades operacionales y restricciones de seguridad bajo las cuales debe operar el sistema, garantizando estabilidad bajo un entorno corporativo de alta concurrencia:

- **RNF-01 Rendimiento y Tiempos de Respuesta:** Al estar desarrollado con una arquitectura desacoplada (React y Node.js), el tiempo de carga inicial no debe exceder los 2 segundos, y las consultas transaccionales de las API RESTful al servidor deben responder en menos de 500 milisegundos en la red interna.
- **RNF-02 Concurrencia y Escalabilidad:** La lógica asíncrona de Node.js debe soportar el acceso y procesamiento concurrente de usuarios concurrentes de las 32 gerencias de la planta de forma simultánea, sin degradación en la persistencia de datos.
- **RNF-03 Seguridad de Datos y Cifrado:** Toda contraseña almacenada en la base de

datos debe estar cifrada mediante algoritmos de hash robustos (como bcrypt). Las comunicaciones lógicas deben viajar protegidas e implementar tokens con tiempo de expiración definido.

- **RNF-04 Usabilidad (UX/UI):** La interfaz estilizada con Tailwind CSS debe seguir principios de diseño intuitivo, permitiendo que un coordinador complete una solicitud en menos de tres pantallas y garantizando la adaptabilidad gráfica a cualquier monitor corporativo.
- **RNF-05 Disponibilidad:** La plataforma web debe mantener un índice de disponibilidad del 99.5% durante las jornadas laborales de la empresa, aislando los mantenimientos programados de la base de datos fuera de las horas pico de producción.
- **RNF-06 Trazabilidad y Auditoría Técnica:** El sistema debe escribir un registro logs de auditoría estricto para cada operación de escritura, actualización o cambio de estatus de una solicitud, identificando de forma unívoca el ID de usuario, fecha, hora y acción realizada.

4.6 Diagramas de Casos de Uso (Estructura Narrativa)

Nota metodológica para el informe impreso: En esta sección de la tesis se deben incrustar los diagramas gráficos exportados de herramientas de modelado como StarUML, Draw.io o Visual Paradigm. A continuación, se detalla la descripción y especificación formal de los casos de uso principales para guiar los flujos lógicos de programación:

Caso de Uso 1: Registrar Solicitud de Compra/Venta

* **Actor Principal:** Coordinador de Área.

* **Precondición:** El Coordinador debe haberse autenticado exitosamente en el sistema.

* **Flujo Principal:** 1) El Coordinador ingresa al módulo de solicitudes. 2) Selecciona "Nueva Solicitud". 3) Rellena el formulario con las especificaciones técnicas del insumo y la prioridad. 4) Presiona "Guardar y Enviar". 5) El sistema valida los datos en el backend, almacena el registro en la base de datos relacional y cambia el estado a "Pendiente por Gerencia".

* **Postcondición:** La solicitud queda registrada en el historial del área y es visible en el panel del Gerente.

Caso de Uso 2: Evaluar y Validar Requerimiento

* **Actor Principal:** Gerente de Área.

* **Precondición:** Existencia de solicitudes en estado "Pendiente por Gerencia" vinculadas a su departamento.

* **Flujo Principal:** 1) El Gerente visualiza la bandeja de entrada de su gerencia. 2) Selecciona una solicitud pendiente para revisar su justificación. 3) Determina la viabilidad técnica y presupuestaria. 4) Selecciona la opción "Aprobar" (o "Rechazar" ingresando observaciones). 5) El sistema altera de forma síncrona el estado de la fila correspondiente en la base de datos.

* **Postcondición:** El flujo avanza hacia el departamento de Almacén/Compras o regresa al

Coordinador para correcciones.

4.7 Diagrama de Clases (Arquitectura del Software)

El Diagrama de Clases modela la estructura lógica orientada a objetos del backend (Node.js/Express) y los esquemas que interactúan con las API RESTful. Representa las entidades del sistema y sus relaciones jerárquicas directas:

- **Clase Usuario (User):** Atributos: id, username, password_hash, email, status. Métodos: login(), logout(), resetPassword(), getRole(). Tiene una relación de asociación de 1 a muchos con el rol y la gerencia.
- **Clase Rol (Role):** Atributos: id, name, description, permissions_array. Define las capacidades lógicas dentro del middleware de la API.
- **Clase Solicitud (Request):** Atributos: id, code, type (compra/venta), date_created, current_status, priority, total_items, justification. Métodos: createRequest(), updateStatus(), addObservation(), archive(). Se asocia con el usuario que crea y el gerente que valida.
- **Clase ItemSolicitud (RequestItem):** Atributos: id, request_id, material_code, quantity, unit_measure, technical_specs. Representa la composición de muchos ítems pertenecientes a una sola solicitud (Relación de composición con Clase Solicitud).
- **Clase HistorialEstado (StatusLog):** Atributos: id, request_id, status_name, user_id, timestamp, remarks. Registra el paso a paso detallado para la auditoría y trazabilidad del proceso logístico.

4.8 Modelo de Entidad-Relación (Base de Datos Normalizada 3FN)

La base de datos relacional se diseñó bajo una estructura normalizada hasta la Tercera Forma Normal (3FN), garantizando la atomicidad de la información y la eliminación de redundancias o dependencias transitorias. A continuación, se detalla el esquema de tablas y relaciones lógicas:

Tabla SQL	Llave Primaria (PK)	Llaves Foráneas (FK) / Relaciones	Campos Principales Atómicos
t_gerencias	id_gerencia (INT)	Ninguna (Tabla Maestra)	nombre_gerencia, codigo_costos, estatus

Tabla SQL	Llave Primaria (PK)	Llaves Foráneas (FK) / Relaciones	Campos Principales Atómicos
t_rols	id_rol (INT)	Ninguna (Tabla Maestra)	nombre_rol, descripcion, permisos_json
t_usuarios	id_usuario (INT)	fk_rol → t_rols(id_rol) fk_gerencia → t_gerencias(id_gerencia)	nombre_usuario, clave_hash, correo_corporativo, estatus_cuenta
t_solicitudes	id_solicitud (BIGINT)	fk_coordinador → t_usuarios(id_usuario) fk_gerencia → t_gerencias(id_gerencia)	codigo_solicitud, tipo_flujo, fecha_creacion, estado_actual, justificacion_texto
t_items_solicitud	id_item (BIGINT)	fk_solicitud → t_solicitudes(id_solicitud)	codigo_material, cantidad_requerida, unidad_medida, especificaciones_tecnicas
t_logs_auditoria	id_log (BIGINT)	fk_solicitud → t_solicitudes(id_solicitud) fk_usuario → t_usuarios(id_usuario)	estado_transicion, observaciones, marca_tiempo, direccion_ip

Lógica de Relaciones Relacionales:

- Gerencias a Usuarios:** Relación de uno a muchos (1:N). Una gerencia engloba múltiples usuarios corporativos, pero un usuario pertenece estrictamente a una sola gerencia.
- Roles a Usuarios:** Relación de uno a muchos (1:N). Un perfil de rol puede asignarse a un conjunto indefinido de cuentas, mientras que cada cuenta posee una identidad de rol activa a la vez dentro de la sesión web.

3. **Usuarios a Solicitudes:** Relación de uno a muchos (1:N). Un coordinador de área puede emitir múltiples solicitudes de compra a lo largo del año operativo, pero cada registro de solicitud guarda traza de un único usuario creador.

4. **Solicitudes a Ítems:** Relación de uno a muchos (1:N) con integridad referencial en cascada (Cascade Delete/Update). Una solicitud logística agrupa un listado atómico de uno o muchos materiales específicos requeridos en planta.